

Arquitetura de Computadores 21/11

PL8

Exercício 1 – Visualize o ficheiro `p108-1.c` com o seguinte código:

```
#include <stdio.h>
#include <omp.h>

main () {
    #pragma omp parallel
    {
        printf ("Hello world\n");
    }
    printf ("That's all, folks!\n");
}
```

- a) Vamos construir o executável. Pretendemos usar o gcc 5.3.0, pelo que deve escrever na sua *shell*:

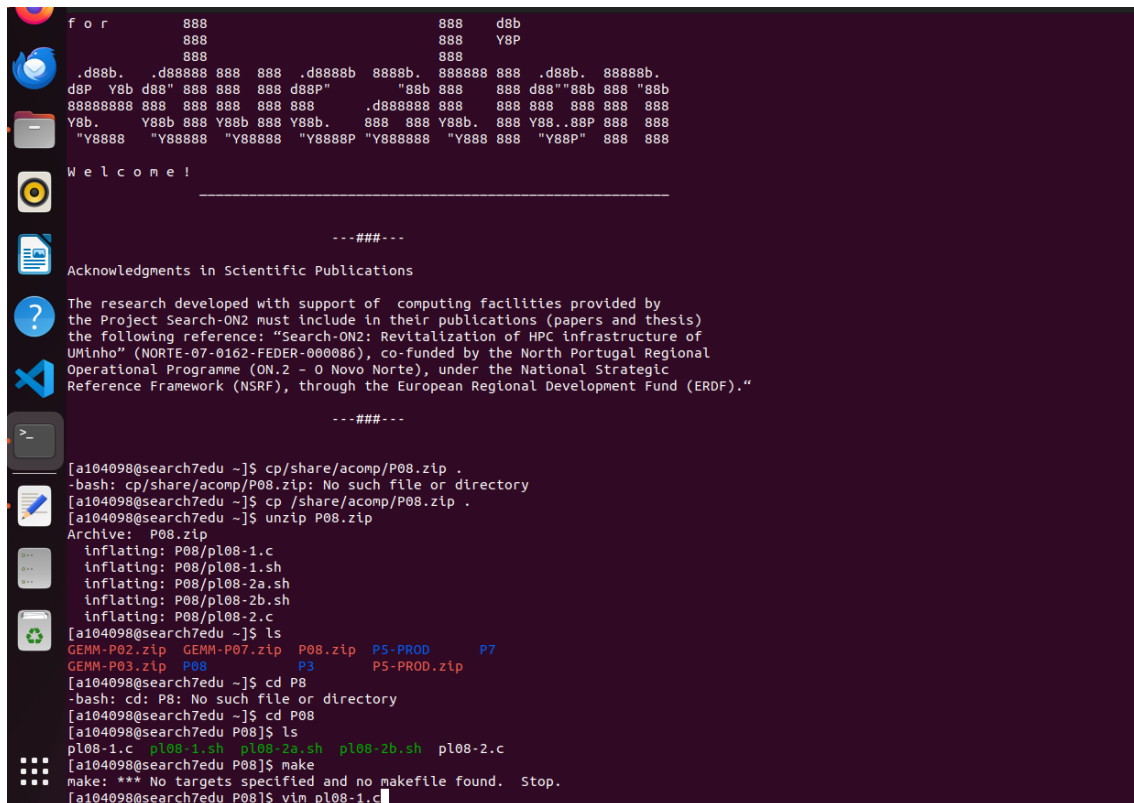
```
module load gcc/5.3.0
```

Compile este código sem usar o OpenMP:

```
gcc -O3 -march=ivybridge p108-1.c -o p108-1
```

Execute este programa (`sbatch p108-1.sh 16`, o argumento da linha de comandos é o número de *threads* OpenMP a criar) e visualize o respectivo *output*. Quantas vezes foi escrita cada uma das frases?

Porquê?



```
f o r      888      888      d8b
            888      888      Y8P
            888      888
.d88b. .d88888 888 888 .d8888b 8888b. 888888 888 .d88b. 88888b.
d8P Y8b d88" 888 888 888 d88P" "88b 888 888 d88"88b 888 "88b
88888888 888 888 888 888 888 .d888888 888 888 888 888 888
Y8b. Y88b 888 Y88b 888 Y88b. 888 888 Y88b. 888 Y88. .88P 888 888
"Y8888 "Y88888 "Y88888 "Y8888P "Y888888 "Y888 888 "Y88P" 888 888

Welcome!

---###---

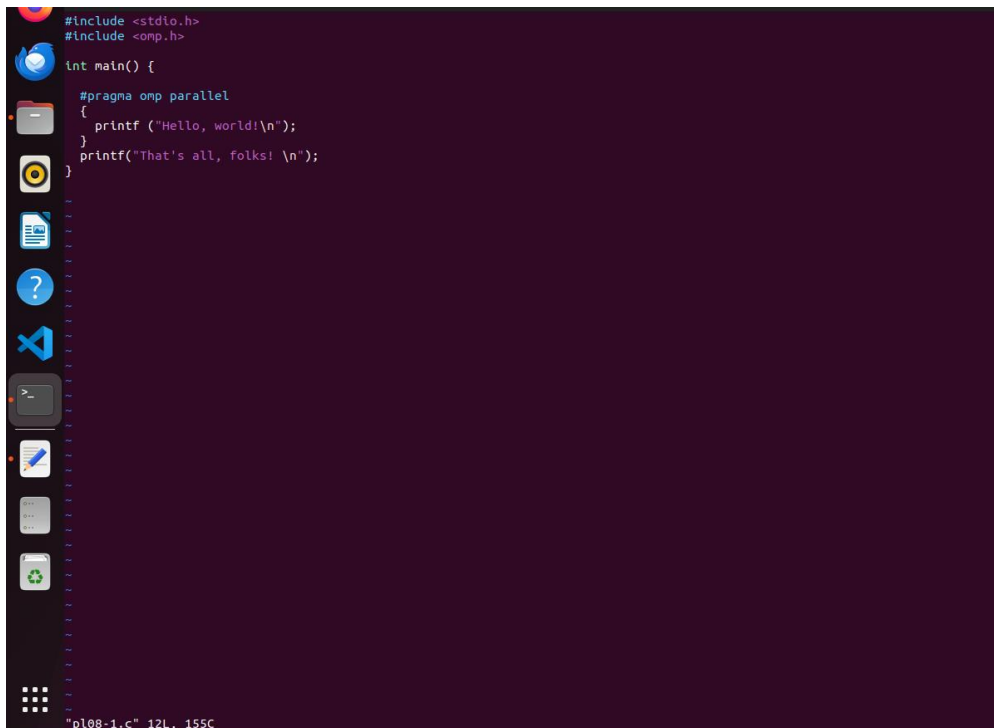
Acknowledgments in Scientific Publications

The research developed with support of computing facilities provided by
the Project Search-ON2 must include in their publications (papers and thesis)
the following reference: "Search-ON2: Revitalization of HPC infrastructure of
UMinho" (NORTE-07-0162-FEDER-000086), co-funded by the North Portugal Regional
Operational Programme (ON.2 - O Novo Norte), under the National Strategic
Reference Framework (NSRF), through the European Regional Development Fund (ERDF).

---###---

[a104098@search7edu ~]$ cp/share/acomp/P08.zip .
-bash: cp/share/acomp/P08.zip: No such file or directory
[a104098@search7edu ~]$ cp /share/acomp/P08.zip .
[a104098@search7edu ~]$ unzip P08.zip
Archive: P08.zip
  inflating: P08/pl08-1.c
  inflating: P08/pl08-1.sh
  inflating: P08/pl08-2a.sh
  inflating: P08/pl08-2b.sh
  inflating: P08/pl08-2.c
[a104098@search7edu ~]$ ls
GEMM-P02.zip  GEMM-P07.zip  P08.zip  P5-PROD  P7
GEMM-P03.zip  P08           P3       P5-PROD.zip
[a104098@search7edu ~]$ cd P8
-bash: cd: P8: No such file or directory
[a104098@search7edu ~]$ cd P08
[a104098@search7edu P08]$ ls
pl08-1.c  pl08-1.sh  pl08-2a.sh  pl08-2b.sh  pl08-2.c
[a104098@search7edu P08]$ make
make: *** No targets specified and no makefile found. Stop.
[a104098@search7edu P08]$ vim pl08-1.c
```

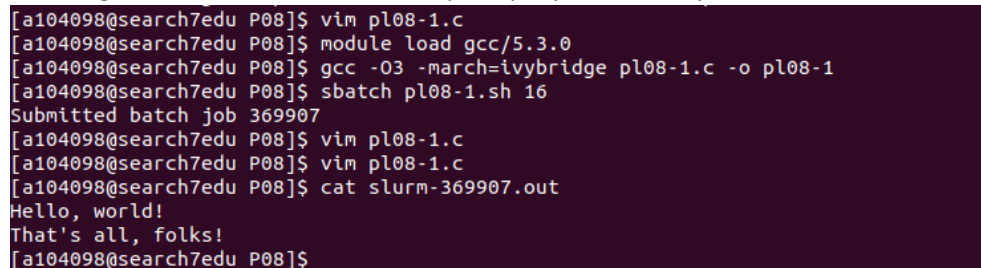
>Enter



```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf ("Hello, world!\n");
    }
    printf("That's all, folks! \n");
}
```

> “i” para escrever e “ESC” para parar de escrever e depois “:q” ou “:wq” (notar que para sair do código tens de escrever “ESC” e dps é que pões o tal “:q”)



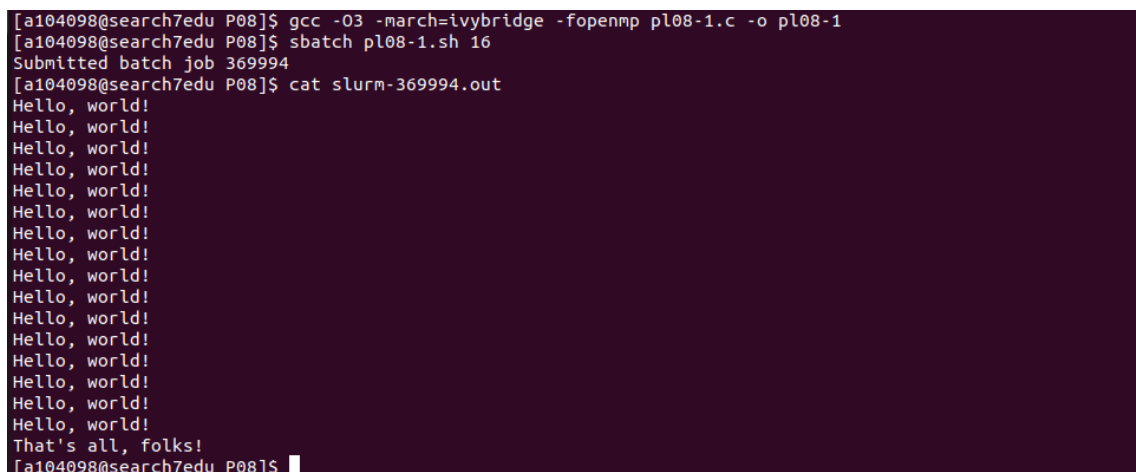
```
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ module load gcc/5.3.0
[a104098@search7edu P08]$ gcc -O3 -march=ivybridge pl08-1.c -o pl08-1
[a104098@search7edu P08]$ sbatch pl08-1.sh 16
Submitted batch job 369907
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ cat slurm-369907.out
Hello, world!
That's all, folks!
[a104098@search7edu P08]$
```

NOTA: o porque de ter tantos “vim” no final é porque tava a testar o “i” e “ESC”, i forgor.

b) Recompile com o comando abaixo:

```
gcc -O3 -march=ivybridge -fopenmp pl08-1.c -o pl08-1
```

Execute este programa (sbatch pl08-1.sh 16) e visualize o respectivo *output*. Quantas vezes foi escrita cada uma das frases? Porquê? Quantas *threads* executaram o bloco paralelo?



```
[a104098@search7edu P08]$ gcc -O3 -march=ivybridge -fopenmp pl08-1.c -o pl08-1
[a104098@search7edu P08]$ sbatch pl08-1.sh 16
Submitted batch job 369994
[a104098@search7edu P08]$ cat slurm-369994.out
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
Hello, world!
That's all, folks!
[a104098@search7edu P08]$
```

- c) A função `int omp_get_thread_num (void)` permite a cada *thread* ter acesso ao seu ID. Declare uma variável "int tid" local ao bloco paralelo, isto é:

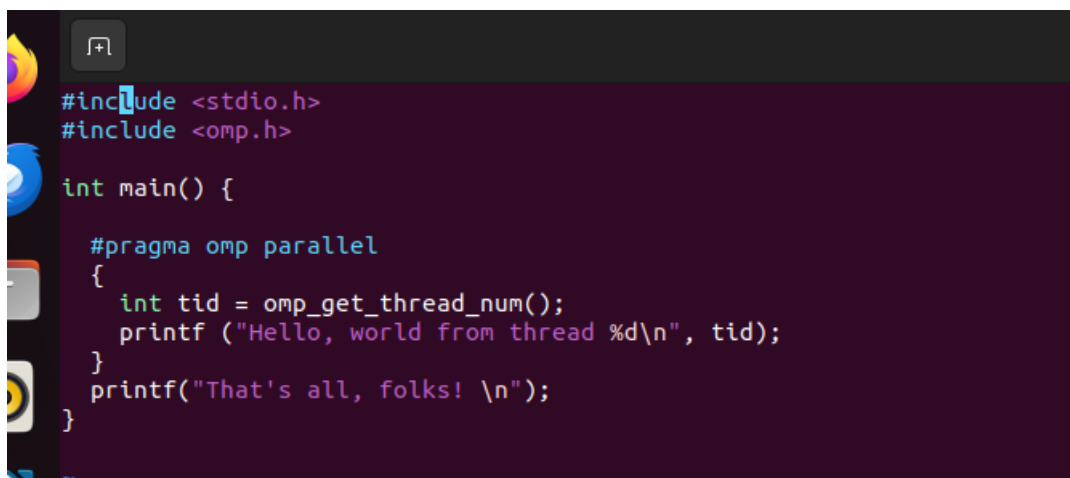
```
#pragma omp parallel
{
    int tid;
    ...
}
```

e, dentro do bloco paralelo, faça cada *thread* ler o seu ID e imprimi-lo:

```
printf ("Hello world from thread %d\n", tid);
```

Note que ao declarar `tid` dentro do contexto (*scope*) do bloco paralelo garante que cada *thread* tem a sua própria instância **privada** da variável `tid`, logo cada *thread* vê apenas a sua própria cópia desta variável. Compile. Execute o programa várias vezes. O *output* é feito sempre pela mesma ordem? Porquê?

➔ Alterar

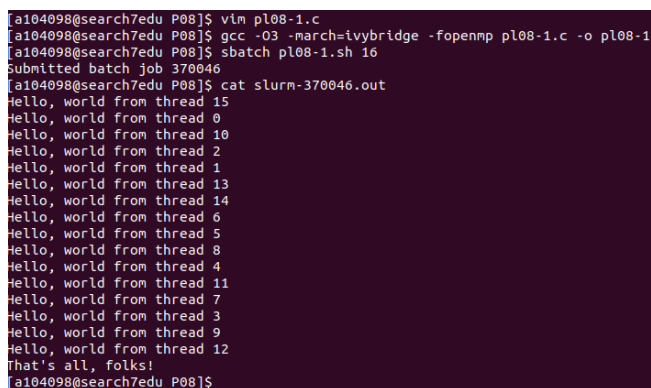


```
#include <stdio.h>
#include <omp.h>

int main() {

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        printf ("Hello, world from thread %d\n", tid);
    }
    printf("That's all, folks! \n");
}
```

output



```
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ gcc -O3 -march=ivybridge -fopenmp pl08-1.c -o pl08-1
[a104098@search7edu P08]$ sbatch pl08-1.sh 16
Submitted batch job 370046
[a104098@search7edu P08]$ cat slurm-370046.out
Hello, world from thread 15
Hello, world from thread 0
Hello, world from thread 10
Hello, world from thread 2
Hello, world from thread 1
Hello, world from thread 13
Hello, world from thread 14
Hello, world from thread 6
Hello, world from thread 5
Hello, world from thread 8
Hello, world from thread 4
Hello, world from thread 11
Hello, world from thread 7
Hello, world from thread 3
Hello, world from thread 9
Hello, world from thread 12
That's all, folks!
[a104098@search7edu P08]$
```

- d) O número de *threads* criadas é, por defeito, igual ao número de processadores vistos pelo Sistema Operativo. Usando a função `int omp_get_num_procs (void)` faça com que a *thread* 0 (e apenas esta) imprima o número de processadores.

Alterar a função

```
#include <stdio.h>
#include <omp.h>

int main() {

    int num_procs = omp_get_num_procs();
    printf("temos %d processadores\n", num_procs);

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        printf ("Hello, world from thread %d\n", tid);
    }
    printf("That's all, folks! \n");
}
```

output

```
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ vim pl08-1.c
[a104098@search7edu P08]$ gcc -O3 -march=ivybridge -fopenmp pl08-1.c -o pl08-1
[a104098@search7edu P08]$ sbatch pl08-1.sh 16
Submitted batch job 370107
[a104098@search7edu P08]$ cat slurm-370107.out
temos 32 processadores
Hello, world from thread 3
Hello, world from thread 9
Hello, world from thread 11
Hello, world from thread 4
Hello, world from thread 13
Hello, world from thread 7
Hello, world from thread 8
Hello, world from thread 15
Hello, world from thread 12
Hello, world from thread 10
Hello, world from thread 2
Hello, world from thread 6
Hello, world from thread 1
Hello, world from thread 5
Hello, world from thread 14
Hello, world from thread 0
That's all, folks!
[a104098@search7edu P08]$
```

- e) A função `int omp_get_num_threads (void)` permite ler o número de *threads* criadas. Deve ser invocada dentro do bloco paralelo. Altere o seu código para que apenas a *thread* 0 leia e imprima o número de *threads* criadas.

```
int main() {

    int num_procs = omp_get_num_procs();
    printf("temos %d processadores\n", num_procs);

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        printf ("Hello, world from thread %d\n", tid);
    }
    printf("That's all, folks! \n");
    //executa só com a thread 0(master)
    #pragma omp master
}

}
```

Exercício 2 – Visualize o ficheiro `p108-2.c` com o código abaixo.

```
#include <stdio.h>
#include <omp.h>
#include <math.h>

#define SIZE 100000000
float a[SIZE], c[SIZE];

main () {
    double T1, T2;
    float f;
    int i;

    for (i=0, f=1.f ; i<SIZE ; i++, f+=1.f) {
        a[i] = f;
    }
    T1 = omp_get_wtime();
    #pragma omp parallel for
    for (i=0 ; i<SIZE ; i++) {
        c[i] = powf (a[i], 3.f) + 10.f / a[i] - 100.f /(a[i] *a[i]);    }
    T2 = omp_get_wtime();
    printf ("That's all, folks! (%.21f secs)\n", T2-T1);
}
```

a) Compile este código usando o comando¹:

```
gcc -O3 -lm -fopenmp -march=ivybridge p108-2.c -o p108-2a
```

Usando o comando `sbatch p108-2a.sh <num_threads>` execute este programa para 1, 2, 4, 8 e 16 threads. Preencha a coluna 2a-Tempo da tabela abaixo.

//como não pus nenhum 1 ou 2 ou 4 ou ...ele presumiu o numero 8

```
[a104098@search7edu P08]$ gcc -O3 -lm -fopenmp -march=ivybridge p108-2.c -o p108-2a
gcc: error: unrecognized command line option '-fopenmp'
[a104098@search7edu P08]$ gcc -O3 -lm -fopenmp -march=ivybridge p108-2.c -o p108-2a
[a104098@search7edu P08]$ sbatch p108-2a.sh <num_threads>
-bash: syntax error near unexpected token `newline'
[a104098@search7edu P08]$ sbatch p108-2a.sh
-bash: sbatch: command not found
[a104098@search7edu P08]$ sbatch p108-2a.sh
Submitted batch job 370246
[a104098@search7edu P08]$ cat slurm-370246
cat: slurm-370246: No such file or directory
[a104098@search7edu P08]$ cat slurm-370246.out
Setting nbr of threads to 8
That's all, folks! (1.01 secs)
[a104098@search7edu P08]$
```

Para 1

```
[a104098@search7edu P08]$ sbatch p108-2a.sh 1
Submitted batch job 370254
[a104098@search7edu P08]$ cat slurm-370254
cat: slurm-370254: No such file or directory
[a104098@search7edu P08]$ cat slurm-370254.out
That's all, folks! (7.89 secs)
[a104098@search7edu P08]$
```

Para 2

```
[a104098@search7edu P08]$ sbatch p108-2a.sh 2
Submitted batch job 370257
[a104098@search7edu P08]$ cat slurm-370257.out
That's all, folks! (3.96 secs)
[a104098@search7edu P08]$
```

Para 4

```
[a104098@search7edu P08]$ sbatch pl08-2a.sh 4
Submitted batch job 370343
[a104098@search7edu P08]$ cat slurm-370343.out
That's all, folks! (1.98 secs)
[a104098@search7edu P08]$
```

Para 8

```
That's all, folks! (1.98 secs)
[a104098@search7edu P08]$ sbatch pl08-2a.sh 8
Submitted batch job 370344
[a104098@search7edu P08]$ cat slurm-370344.out
That's all, folks! (1.03 secs)
[a104098@search7edu P08]$
```

Para 16

```
[a104098@search7edu P08]$ sbatch pl08-2a.sh 16
Submitted batch job 370345
[a104098@search7edu P08]$ cat slurm-370345.out
That's all, folks! (0.51 secs)
[a104098@search7edu P08]$
```